

Object is subclassed and given a property called “greeting” and an instance of that object is to the QML root context, where we can access it from the QML file.

This is the definition of our “Hello” class – we provide a getter method for the greeting, and return a unicode string “Hello, Digit!” in it

- `app = QApplication(sys.argv)`

`QApplication` is needed by every Qt application and handles command-line arguments, sets up the graphics system and does other initializations.

- `context = view.rootContext()`
- `context.setContextProperty('hello', hello)`
- `view.setSource(__file__.replace('.py', '.qml'))`

For QML to be able to access our “hello” object, we need to expose it as a context property to the view’s root context – this is done using `setContextProperty`.

The QML file is then loaded and displayed in the view.

- `view.show()`
- `app.exec_()`

And finally, here our application starts: We always have to call the `show()` method on the view, or otherwise it won’t be shown on the screen.

In order to start the Qt main loop and process events, we have to call `app.exec_()`, so the application does not quit. This is very important.

The entire code then looks like :-

- `import sys`
- 
- `from PySide.QtCore import *`
- `from PySide.QtGui import *`
- `from PySide.QtDeclarative import *`
- `class Hello(QObject):`